

Chapter 6 - Summary and Sample Questions

Autoregressive modeling: the core probabilistic idea

An **autoregressive** model defines a probability distribution over a sequence by factorizing it into a product of conditional probabilities:

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{<t}).$$

This equation is important because it turns the difficult task of modeling an entire sequence at once into a sequence of **next-step prediction** problems. In language modeling, for example, the model learns to predict the next token from the tokens that came before it. Training is therefore closely tied to the probabilistic structure of the model: learning to predict the next token well is the same as improving the likelihood of the training data.

This formulation also explains how generation works. At inference time, the model starts with an initial prompt, predicts a distribution over the next token, samples or selects one token, appends it to the context, and repeats. So the same factorization supports both **training** and **generation**.

2. RNNs and LSTMs: early sequence models

Before Transformers, sequence modeling was dominated by **Recurrent Neural Networks (RNNs)** and later **Long Short-Term Memory (LSTM)** networks. An RNN processes a sequence one step at a time, updating a hidden state:

$$h_t = f(h_{t-1}, x_t).$$

The hidden state acts as a running summary of the past. This is elegant, but it creates a bottleneck: all relevant earlier information must be compressed into a single evolving vector. As sequences become longer, this compression becomes harder, and learning long-range dependencies becomes unreliable.

LSTMs improve on this by introducing gated mechanisms that control what information is written, kept, and exposed. Conceptually, the gates let the model decide when to preserve older information and when to overwrite it. This helps with vanishing-gradient problems and makes longer dependencies more manageable than in plain RNNs. However, LSTMs still process tokens **sequentially**, which limits parallelism and can still make very long-range reasoning difficult.

Training recurrent models requires **Backpropagation Through Time (BPTT)**. Since the same recurrent computation is reused across many time steps, gradients must be propagated through a long chain of dependencies. This makes optimization sensitive to exploding or vanishing gradients and highlights an important lesson: even when the model definition is simple, the optimization path may be difficult.

Why attention changes the picture

The central idea behind the Transformer is that a token should not rely only on a compressed hidden state from the immediate previous step. Instead, it should be able to look back at the entire relevant context and determine **which earlier tokens matter most** for constructing its current representation.

This is the role of **attention**. In self-attention, each token produces three vectors:

- a **query** that represents what information the token is looking for,
- a **key** that represents what kind of information each token offers,
- a **value** that contains the content to be aggregated.

The compatibility between token i and token j is measured by a score such as:

$$s_{ij} = \frac{q_i k_j^T}{\sqrt{d_k}}.$$

These scores are normalized with a softmax to produce attention weights, and the output for token i becomes a weighted combination of value vectors:

$$\alpha_{ij} = \text{softmax}_j(s_{ij}),$$
$$\text{out}_i = \sum_j \alpha_{ij} v_j.$$

This gives each token a **contextualized representation**. A word is no longer represented only by its standalone embedding, but by a representation that depends on the surrounding tokens and the task of the current layer.

The use of separate query, key, and value projections is conceptually important. The model not only decides **where to look** but also **what content to retrieve** from those locations. Attention is therefore more than a weighted average; it is a learned content-based retrieval mechanism.

The Transformer block

A Transformer does not consist of attention alone. A standard block usually contains:

1. **Multi-head self-attention**
2. A **position-wise feed-forward network (FFN)**
3. **Residual connections**
4. **Layer normalization**

Multi-head attention means the model runs several attention mechanisms in parallel, each with different learned projections. This allows the model to capture different kinds of relationships at the same time, such as short-range syntactic cues, long-range semantic references, or positional patterns.

The **FFN** is applied independently to each token after attention. Attention mixes information **across tokens**, while the FFN transforms information **within each token representation**. This combination is powerful: attention handles interaction across positions, and the FFN increases local representational capacity.

Residual connections help preserve information and support gradient flow, while **LayerNorm** stabilizes training by normalizing features within each token representation. Together, these design choices make deep Transformer stacks trainable in practice.

Because self-attention compares tokens directly, Transformers avoid the single-state bottleneck of RNNs and allow much greater parallelism during training. This is one of the main reasons they became dominant.

Positional information and causal structure

Self-attention by itself is permutation-invariant: without additional structure, it does not know which token came first or last. That is why Transformers need **positional information**, added through positional encodings or learned positional embeddings.

For **decoder-only** language models such as GPT, attention must also obey the autoregressive constraint: token t may only attend to tokens up to position t , not future tokens. This is enforced by **causal masking**. Without such masking, the model could cheat during training by directly inspecting future tokens it is supposed to predict.

Tokenization and BPE

Language models do not usually operate on raw words. Instead, they operate on **tokens** drawn from a finite vocabulary. One practical challenge is that word-level vocabularies are too rigid and character-level models can make sequences unnecessarily long. **Byte-Pair Encoding (BPE)** provides a compromise by learning frequent subword units.

BPE starts from smaller units and repeatedly merges common symbol pairs. This allows the tokenizer to represent frequent words compactly while still handling rare or unseen words through smaller pieces. Conceptually, tokenization is not a minor preprocessing step; it changes the length of sequences, the granularity of meaning, the efficiency of training, and the kinds of patterns the model can learn.

GPT-style training and the larger Transformer family

A GPT-style model is a **decoder-only Transformer** trained with the next-token objective. Given an input prefix, the model outputs logits for the next token at every position. A softmax converts logits into probabilities, and training minimizes cross-entropy against the true next token. This is directly aligned with the autoregressive factorization of sequence probability.

This gives rise to a broader family of Transformer architectures:

- **Encoder-only** models learn bidirectional contextual representations and are strong for understanding tasks such as classification, tagging, and retrieval.
- **Decoder-only** models are naturally suited for generation because they are trained autoregressively.
- **Encoder-decoder** models separate source encoding from target generation and are especially useful for sequence-to-sequence tasks such as translation and summarization.

The broader lesson is that modern sequence modeling is not just about replacing one neural network with another. It reflects a shift in how context is represented: from a compressed sequential state to a flexible, learned pattern of content-based interactions across tokens.

Final Exam Conceptual Questions with Sample Answers

These questions are designed to be **self-contained, conceptual**, and suitable for an advanced final exam. They emphasize understanding, interpretation, and synthesis rather than programming.

Question 1

An autoregressive model writes

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{<t}).$$

Explain why this is both a probabilistic statement and a practical training strategy.

Sample Answer

It is a probabilistic statement because it expresses the joint probability of the whole sequence in terms of conditional distributions. The model is therefore assigning likelihood to complete sequences, not merely making isolated predictions.

It is also a practical training strategy because maximizing the joint likelihood becomes equivalent to training the model to predict each next token from its prefix. This converts sequence modeling into a large collection of supervised prediction problems, one at each position in the sequence.

The factorization also explains generation: once the conditionals are learned, the model can generate by repeatedly predicting and appending the next token.

Question 2

In self-attention, why do we need separate query, key, and value projections? Why not let one vector play all three roles?

Sample Answer

The query represents what a token is currently looking for, the key represents what each token offers for matching, and the value represents the content to be retrieved once a match is made. Keeping these roles separate gives the model more expressive flexibility.

If one vector had to serve all three roles, the model would have less freedom to distinguish matching behavior from content storage. Separate projections allow the model to learn one geometry for deciding relevance and another for representing information to be passed forward.

This separation is one reason attention can behave like learned retrieval rather than simple similarity averaging.

Question 3

The output of attention is a weighted sum of value vectors. Why is the value vector conceptually necessary once the model has already computed relevance weights?

Sample Answer

The relevance weights only indicate which positions matter and by how much. They do not specify what information should actually be extracted from those positions.

The value vectors provide the content that is aggregated after the relevance pattern has been decided. This separation means a token can attend strongly to a position because it is relevant, while still retrieving a transformed representation of the information stored there.

Without values, attention would know where to look but not have a flexible mechanism for deciding what representation to pull back from that location.

Question 4

Why can multi-head attention be more expressive than single-head attention even if both operate on the same input sequence?

Sample Answer

Different heads use different learned projections, so they can focus on different types of relationships at the same time. One head might track local syntactic structure, another might follow long-range references, and another might attend to positional or semantic patterns.

A single head must compress all these possibilities into one attention map and one value aggregation process. Multi-head attention distributes the work across parallel subspaces, allowing the model to represent richer interaction patterns.

The advantage is not just more parameters, but structured diversity in how context is analyzed.

Question 5

Why is causal masking essential in a decoder-only language model? What would happen conceptually if it were removed during training?

Sample Answer

A decoder-only language model is supposed to model

$$p(x_t \mid x_{<t}).$$

Causal masking enforces this by preventing token t from attending to future tokens. If the mask were removed during training, the model could directly use the answer token it is supposed to predict.

This would make training deceptively easy but would destroy the intended probabilistic interpretation. The model would no longer be learning a valid autoregressive conditional distribution, and the resulting training objective would not match generation-time constraints.